

Behavior Belongs in Cells: A Standalone Formalization of the Substrate-as-Cell Anti-Pattern in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone failure-mode specification, the anti-pattern in which the substrate is configured to execute behavior — the inverse boundary-collapse to the cell-as-substrate failure that holds substrate-relevant state outside the substrate.

Abstract

The CKS pattern commits to a categorical separation between a substrate that holds state and cells that execute behavior over that state. The substrate-as-cell anti-pattern is the deployment configuration in which the substrate has acquired execution responsibilities: triggers that run on writes, stored procedures that implement coordination logic, computed columns that derive content through substrate-resident transformation, active rules that fire when conditions are met. This note specifies the anti-pattern as four operational components, identifies the CKS commitments it violates (primary: the substrate-cell boundary, through cells-execute-behavior; cascade: retraceability, determinism, mediator role, cost scaling), states the architectural correction, distinguishes the failure from four adjacent legitimate substrate features, and provides an operational test with three sharpening properties. The substrate-as-cell anti-pattern is the inverse of the cell-as-substrate anti-pattern; together they cover the bidirectional collapse of the substrate-cell boundary.

1. Why a precise formalization is needed

The substrate-cell boundary commits to two reciprocal architectural facts: substrate holds state without executing behavior, and cells execute behavior over the state the substrate holds. The inverse failure modes follow. If a deployment has cells holding substrate-relevant state outside the substrate, the boundary collapses in one direction — the cell-as-substrate failure. If a deployment has substrate executing behavior, the boundary collapses in the opposite direction. This note formalizes the second failure as a standalone publication.

The formalization is necessary now for three reasons. *Structurally*, a boundary stated only as positive commitments is incompletely specified without a parallel statement of the failure modes that violate it;

the cell-as-substrate failure has been formalized separately, and without a parallel formalization of substrate-as-cell the boundary's failure modes are described asymmetrically. *Operationally*, database systems used as CKS substrates commonly support embedded execution as a feature — triggers, stored procedures, computed columns, materialized views with computation — and the engineering pattern of placing logic close to data extends naturally to coordination logic when such a substrate hosts coordination state, producing the substrate-as-cell shape without the architectural change being named. *Downstream*, several other CKS anti-patterns assume the substrate-cell boundary holds at the layer they describe; substrate-resident execution masks where state lives, evades cell mediation, and confers ungoverned-writer status on adjacent components by indirection, so the boundary failure must be named upstream of the failures it compounds.

2. The anti-pattern, defined as four operational components

A deployment exhibits the substrate-as-cell anti-pattern when its substrate is configured to execute behavior — to run code, evaluate logic, transform content, or fire reactive responses — rather than holding state passively. The architectural failure has four operational components, and a deployment exhibits the anti-pattern partially when any one is present and fully when all four are present.

(a) Substrate-embedded execution triggers. The substrate is configured with triggers that execute on writes: when state changes, embedded logic runs as part of the write operation. Concrete instances include database triggers, change-data-capture handlers attached to substrate state, watch handlers that fire on substrate updates, and equivalent mechanisms. The architectural commitment that cells execute behavior fails because triggers run behavior at substrate operations, outside any cell's bounded execution scope.

(b) Substrate-resident active rules with execution semantics. The substrate hosts active rules — event-condition-action specifications that fire on conditions and run actions when their conditions are met. The rules are not held as substrate content (passive specifications cells consult under orchestration) but as substrate-resident execution units. The distinction between rules-as-content (which cells read and interpret under their own orchestration) and rules-as-execution-units (which run themselves at the substrate layer) collapses, and the substrate has acquired behavior the cell layer is responsible for.

(c) Substrate-side data transformation logic. The substrate contains logic that transforms content: computed columns that derive values from other columns, view definitions that include computation beyond projection or filtering, substrate-resident functions invoked during reads or writes. The transformations may be transparent to readers — the read returns a value as if it were stored — but the substrate is operationally executing logic each time the value is produced. The substrate's commitment to passive storage fails because the storage layer is computing.

(d) Substrate-as-event-source for reactive behaviors. The substrate operates as an event source publishing state changes; subscribers, which may be cells or adjacent components, react to the published events. While event-driven coordination is possible at the cell layer (a cell may be invoked because an orchestration rule names a substrate condition as its trigger), substrate-as-event-source

positions the substrate itself as an active component with publication semantics. The substrate is no longer a passive store that cells read; it is an active participant whose state changes drive the system's behavior.

A deployment that exhibits any one of the four components has compromised the boundary in the direction the component names; a deployment exhibiting all four has positioned its substrate as an executor of behavior across writes, rules, derivations, and reactivity.

3. The CKS commitments violated

The substrate-as-cell anti-pattern violates the substrate-cell boundary directly and produces cascade violations of four further commitments.

Primary violation: the substrate-cell boundary, through cells-execute-behavior. The boundary commits to substrate holding state without executing behavior, and cells executing behavior over substrate state. Substrate-as-cell fails the boundary in the substrate-acquires-behavior direction: the substrate now executes behavior the architecture locates in cells. Where cell-as-substrate compromises the cells-do-not-hold-substrate-state side of the boundary, substrate-as-cell compromises the substrate-does-not-execute-behavior side. The two failures are inverse-symmetric and together cover the bidirectional collapse.

Cascade: path retraceability. The retraceability commitment depends on every substrate write being recorded with the cell that produced it, the rule that authorized the cell, the rationale, and the antecedents. Substrate-resident execution has no cell to attribute writes to and no orchestration rule to record; the trail breaks at every substrate execution boundary, and audit cannot reconstruct what happened and why.

Cascade: the determinism contract. Substrate-executed behavior may depend on context that is not part of substrate state — invoking session, transaction isolation, runtime environment, the order in which cascading triggers fire. The behavior's output is then a function of substrate-execution context invisible to readers operating over substrate content. Hidden non-determinism enters the system at the substrate layer, where the architecture commits to determinism most strongly.

Cascade: AI-as-substrate-mediator. The mediator role places the LLM inside cells, reading from and writing to the substrate under human-authored orchestration rules. When the substrate executes coordination logic, the mediated path is bypassed: the work happens at the substrate layer without the LLM's involvement and without an orchestration rule's authorization.

Cascade: linear-cost scaling. The linear-cost commitment holds that storage grows with content but governance, cell execution, and LLM inference do not grow with substrate size. Substrate-resident execution introduces a cost the commitment does not account for: substrate operations now have execution semantics whose cost may grow with substrate size — triggers that scan related rows, stored procedures whose complexity depends on substrate scale, computed columns requiring substrate-wide recomputation. The contract holds only when substrate operations are state-modifying without execution; substrate-as-cell removes the precondition.

Extended: AI-as-mediator at every layer. The composition requirement that the mediator role hold at every layer is extended-violated when the substrate layer itself executes coordination logic without mediation. The same architectural fact that makes adjacent ungoverned writers an anti-pattern makes substrate-resident execution one: behavior that affects coordination state runs without passing through a cell under a rule.

Partial: rules-as-content source-of-truth. The substrate's authority over what rules apply commits to rules being addressable, inspectable substrate content. When active rules are substrate-resident execution units rather than substrate content, the content/execution distinction blurs: changing a rule may require modifying substrate-resident execution logic, and rule modification operates outside the inspect-modify-override rights that apply to substrate content as content.

4. The failure mode

The cascade implications named in §3 manifest, in deployments that drift into substrate-as-cell, as a recognizable pattern. The locus of behavior becomes ambiguous: some logic runs in cells under orchestration rules, some runs at the substrate layer through triggers, procedures, computed columns, or active rules, and a reader asking "where is this logic?" finds two answers. The architectural commitment that behavior lives in cells no longer holds as a single clean fact about the deployment.

Two further consequences compound the boundary failure. First, substrate-resident logic drifts on a separate cadence from substrate content. Triggers, stored procedures, and computed-column formulas are substrate-resident execution rather than substrate content; modifications to them may not register as substrate-content modifications and may be governed under processes distinct from those that apply to substrate content. Second, substrate-embedded triggers can fire other triggers, producing emergent behavior whose architectural unit is substrate-internal. Path retraceability over the cascade is particularly compromised because the cascade's intermediate states are substrate-internal events, not cell executions.

5. The architectural correction

The architectural correction operates through three reciprocal commitments held together: substrate as passive store, behavior in cells, and computed values via derived view rather than substrate-resident computation.

The substrate must hold state without executing behavior. Substrate operations are state-modifying — set, get, update, delete, with integrity constraints rejecting structurally invalid writes — and do not have execution semantics. Triggers, stored procedures implementing coordination, computed columns whose values are derived through substrate-side computation, and active rules with execution semantics are removed from substrate configuration; what remains is storage, indexing, and structural validation.

Behavior lives in cells. Logic that previously ran in substrate-resident execution moves into cells, where it executes under orchestration rules with the LLM as mediator where mediation applies, and with provenance recorded for every write. A computation that lived in a database trigger becomes a cell that executes when its rule's condition is met; a stored procedure becomes one or more cells whose

orchestration rules define when they run and what they read and write; a computed column whose value derives from substrate content becomes a cell that produces the derived content as substrate writes under a rule.

Computed values, where performance considerations argue for pre-computation, are produced through Pattern B — adjacent component as derived view of substrate content. The derived view's computation lives in the adjacent component; the view is regeneratable from substrate state at any time; the substrate remains authoritative; writes do not flow back from the view to the substrate. This is the architecturally legitimate way to capture the performance benefit substrate-resident computation appears to offer.

A correctly architected deployment additionally audits its substrate configuration to confirm the absence of substrate-resident execution, preserves integrity constraints and indexes as legitimate substrate features, and distinguishes materialized views holding projection or filtering of substrate content (legitimate) from materialized views whose maintenance involves substrate-resident computation (substrate-as-cell).

6. What the anti-pattern is NOT

Four adjacent substrate features are commonly conflated with substrate-as-cell. Naming each is what makes the anti-pattern auditable.

Not integrity constraints. Foreign keys, uniqueness constraints, non-null constraints, and structural schema validation prevent invalid state from entering the substrate. They reject writes that would violate structural commitments; they do not execute coordination logic. Integrity constraints are legitimate substrate features and do not constitute the anti-pattern.

Not indexes. B-tree indexes, hash indexes, vector indexes for retrieval, full-text indexes — substrate features that support read performance over substrate content — accelerate access to content that is already there; they do not derive new content. Indexes are legitimate substrate features.

Not derived views per Pattern B. Substrate-derived projections, summaries, or indices held in adjacent components and treated as non-authoritative are the architecturally legitimate way to capture pre-computed content. The computation lives outside the substrate; the substrate remains authoritative; the view is regeneratable from substrate state. A derived view per Pattern B is the correct shape for what substrate-side computation incorrectly tries to provide.

Not schema validation. Type checking, format verification, and structural validation that reject malformed writes operate on the structural properties of substrate content; they do not execute coordination logic. A schema that enforces "this field is a date in ISO 8601 format" is rejecting structurally invalid writes; a schema that derives values, evaluates conditions, or fires reactive behaviors is doing something different and exhibits the anti-pattern.

The distinguishing line in each case is the same: storage-layer features that preserve substrate's passive role are legitimate; execution-layer features that compute, derive, or react are the failure.

7. Operational test, and why naming this anti-pattern matters

A deployment exhibits substrate-as-cell if any of the following are true at any time during the deployment's existence:

1. Substrate is configured with triggers, change-data-capture handlers, watch handlers, or equivalent mechanisms that execute logic on writes.
2. Substrate hosts stored procedures or substrate-resident functions that implement coordination logic beyond passive storage.
3. Substrate has computed columns or substrate-side data transformations that derive values through substrate-resident computation.
4. Substrate hosts active rules with execution semantics that fire on conditions and run actions at substrate operations.

Three sharpening properties convert the test into a deployment review:

Passive-storage test. Substrate operations are state-modifying without execution semantics. Examining substrate configuration reveals only storage-layer features — set, get, update, delete, integrity constraints, indexes, structural validation; execution-layer features are absent.

Behavior-locus test. Coordination behavior, derived computation, and reactive responses execute in cells under orchestration rules, not in substrate. Tracing where behavior runs in the deployment yields a single architectural answer: in cells, never in substrate.

Substrate-operations-determinism test. Substrate operations are deterministic from substrate state alone. Examining substrate operation outputs given identical inputs shows no dependence on substrate-runtime context — invoking session, transaction state, server environment, or substrate-execution variables.

A deployment that fails any of (1)–(4) and any of the three sharpening properties exhibits the anti-pattern; §5 specifies the operational changes required.

The one-sentence test is short enough to apply at design review: *if a deployment's substrate is configured to execute behavior — through triggers, stored procedures, computed columns with substrate-resident logic, active rules, or reactive substrate operations — and coordination behavior runs at substrate operations rather than in cells under orchestration rules, the deployment exhibits substrate-as-cell.*

The anti-pattern needs to be named because the engineering pattern that produces it is familiar and the architectural consequence is easy to miss. Embedded execution is a routine feature of the database systems often used as substrates, and "we use database triggers for coordination logic" reads as standard engineering practice rather than as the failure mode it is. Naming the four operational components, the cascade implications, and the architectural correction makes the drift visible at the moment a deployment chooses between locating logic in substrate and locating it in cells. Together with the cell-as-substrate anti-pattern, substrate-as-cell completes the bidirectional account of how the substrate-cell boundary fails.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Behavior Belongs in Cells: A Standalone Formalization of the Substrate-as-Cell Anti-Pattern in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.